Memòria de la Pràctica: Mr. J System



Sistemes Operatius

Laura Cuenca – Laura.cuenca@students.salle.url.edu

Curs 2024-25

Índex

1. Introducció 3

1. Disseny 5

2.1. Estructures de Dades 6

1. ConfigBase 6

2. ConfigGotham, ConfigFleck i ConfigWorker 6

3. TaskMetadata 7

4. TaskStatus 7

5. DistorsioStatus 7

6. DistorsioArgs 8

7. Trama 8

2.2. Estratègia de concurrència adoptada per mòdul 9

Gotham 9

Workers (Harley / Enigma) 12

Fleck 17

2.3. Gestió de memòria compartida 20

Estructura de la memòria compartida 20

Ampliació dinàmica amb mremap 20

Sincronització d'accés 21

Persistència entre Workers 21

2.4. Modularització del codi 21

Estructura de carpetes 22

Llibreries compartides 22

2.5. Funcionalitats opcionals implementades 24

3. Problemes observats i solucions 25

4. Diagrames de Flux 28

Connexió inicial de Fleck amb Gotham i dels Workers amb Gotham..... 28

Gestió de Distorsió..... 29

5. Estimació temporal..... 31

6. Conclusions i propostes de millora..... 32

7. Explicació noms dels processos..... 34

8. Bibliografia..... 35

1. Introducció

La pràctica implementa un sistema distribuït amb arquitectura client-servidor orientada a connexions TCP/IP, dissenyat per gestionar de forma concurrent peticions de distorsió de fitxers provinents de diversos clients. L'objectiu principal és construir una infraestructura robusta, eficient i escalable capaç de rebre fitxers, processar-los segons el seu tipus (multimèdia o text) i retornar-ne la versió distorsionada al client.

El sistema se centra en la gestió **concurrent de múltiples peticions**, garantint que cada procés pugui continuar funcionant fins i tot si altres processos es desconnecten, fallen o es sobrecarreguen. Aquest enfocament està inspirat en escenaris reals de sistemes operatius, com servidors de fitxers o aplicacions distribuïdes amb múltiples punts de fallada potencials.

Els components del sistema són els següents:

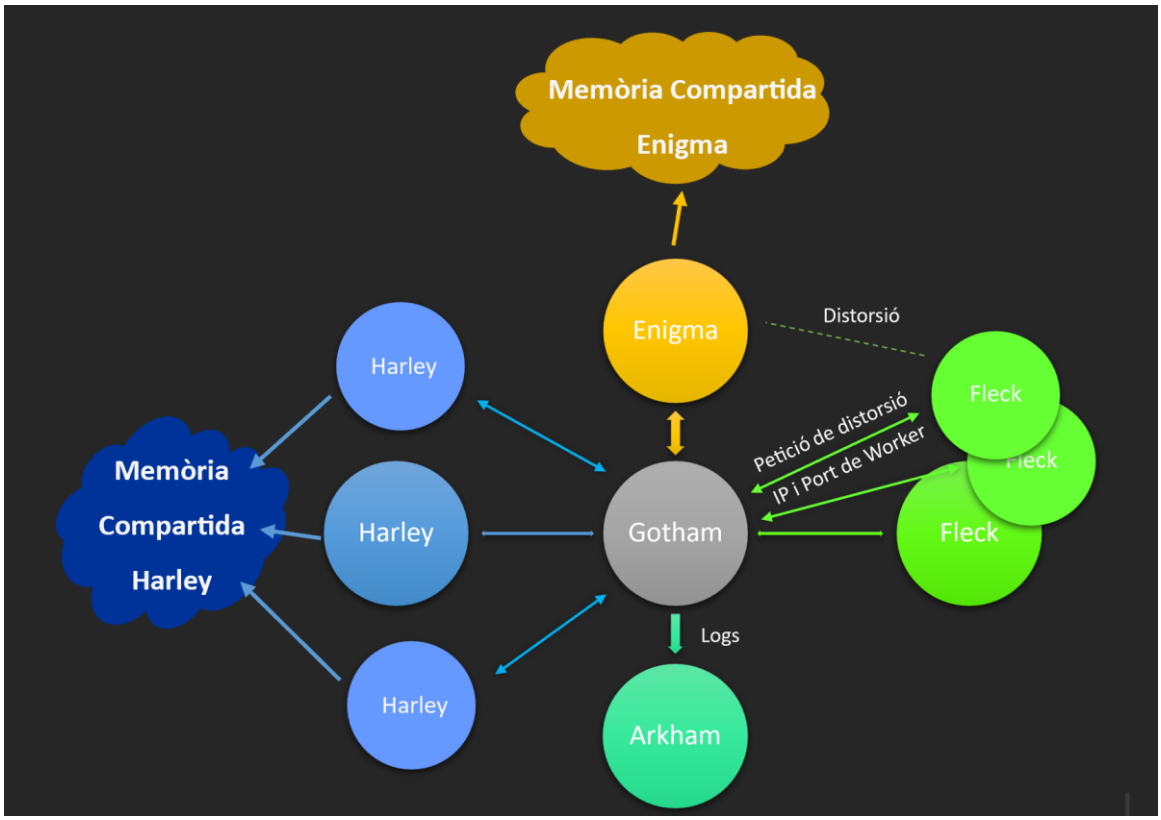
- **Fleck**: procés client que permet a l'usuari demanar distorsions de fitxers, fer-ne el seguiment i visualitzar l'estat de les operacions. Està preparat per gestionar distorsions simultànies i reaccionar davant errors de connexió o fallades de workers. Ofereix una interfície interactiva en forma de shell i gestiona connexions amb Gotham i els workers.
- **Gotham**: servidor central encarregat de coordinar la comunicació entre els clients i els treballadors. Registra connexions entrants, redirigeix peticions de distorsió al worker corresponent segons el tipus de fitxer, i gestiona la promoció de workers principals i la detecció de caigudes mitjançant **heartbeats**.
- **Workers (Harley i Enigma)**: processos encarregats d'efectuar la distorsió dels fitxers. **Harley** tracta fitxers multimèdia (.jpg, .png, .wav) amb funcions de compressió, mentre que **Enigma** tracta fitxers de text eliminant paraules curtes segons un llindar. Els workers poden ser reassignats i treballen amb memòria compartida per garantir persistència de tasques i robustesa davant caigudes.
- **Arkham**: procés fill de Gotham encarregat de registrar esdeveniments importants del sistema en un fitxer de log compartit (logs.txt). Rep missatges per pipe i actua com a logger centralitzat.

El flux de dades segueix la ruta:

2. L’usuari, des de Fleck, demana una distorsió.
3. Fleck sol·licita un worker a Gotham.
4. Gotham assigna un worker disponible del tipus adequat.
5. Fleck envia el fitxer i rep la versió distorsionada.
6. Gotham i Arkham registren l’activitat i supervisen l’estat dels components.

Aquesta arquitectura assegura modularitat, tolerància a fallades, i capacitat per escalar amb més clients i treballadors en paral·lel. La separació de responsabilitats entre components i la utilització de concurrència real permeten un comportament fluid fins i tot en situacions amb múltiples connexions actives.

1. Disseny



El sistema està dissenyat, tal i com es veu al diagrama, seguint una **arquitectura distribuïda modular** basada en processos independents que es comuniquen mitjançant **sockets TCP/IP**. Cada component té responsabilitats ben definides i comunica amb els altres mitjançant un protocol de trames unificat. Aquesta arquitectura afavoreix la separació de responsabilitats, la robustesa davant fallades i la possibilitat de treballar de manera concurrent i escalable., amb els següents components:

- **Gotham**:
  - **S**ervidor central que coordina, assigna treballadors i monitoritza el sistema.
- **Fleck**:
  - Permet a l'usuari sol·licitar distorsions i supervisar-ne l'estat.
- **Harley i Enigma**:
  - Processen fitxers multimèdia i de text, respectivament.

    - Implementen mecanismes de recuperació amb memòria compartida per garantir robustesa.
- **Arkham**:
    - Registra esdeveniments i errors en temps real en un fitxer únic (logs.txt).

El flux de comunicación principal seria:

1. Fleck sol·licita una distorsió a Gotham.
2. Gotham assigna un worker (Harley o Enigma) segons el tipus de fitxer.
3. Fleck s'hi connecta directament, envia el fitxer, espera la resposta i comprova la seva integritat.
4. Arkham, que rep missatges per pipe de Gotham, registra els esdeveniments més rellevants al fitxer logs.txt.

2.1. Estructures de Dades

En el disseny del sistema, les estructures de dades han tingut un paper fonamental per poder organitzar, gestionar i comunicar la informació entre processos. A continuació es detallen les estructures creades, amb justificació de cada camp i el seu ús en el sistema:

1. ConfigBase

Aquesta estructura encapsula els paràmetres bàsics de configuració de qualsevol instància del sistema (ja sigui Gotham, Fleck o un Worker). Inclou:

- InstanceType type: indica si és Gotham, Fleck, Enigma o Harley.
- char *ip i int port: defineixen on escolta aquesta instància.
- int socket_fd: descriptor del socket actiu.

Justificació: permet agrupar la informació bàsica comuna de manera reutilitzable i passar-la entre funcions com inicialitzar_connexio_gotham o enviar_trama.

2. ConfigGotham, ConfigFleck i ConfigWorker

Són estructures derivades de ConfigBase que afegeixen camps específics:

- ConfigGotham: conté també l'IP i port on escolta els workers.
- ConfigFleck: afegeix un fleck_id, un username, el work_dir i el last_heartbeat.

- ConfigWorker: afegeix worker_id, work_dir, IP/port d'escolta a Fleck, i si és el worker principal i el last_heartbeat.

Justificació: permet separar clarament la configuració i estat de cada tipus d'instància, facilitant modularitat i mantenibilitat.

3. TaskMetadata

Aquesta estructura defineix les metadades de cada tasca de distorsió:

- file_name: nom del fitxer.
- total_size: mida original o distorsionada del fitxer.
- processed_offset: bytes ja processats.
- bytes_sent: bytes enviats al Fleck.
- status: estat de la tasca (usant l'enum TaskStatus).
- checksum: per comprovar integritat.
- factor: nivell de compressió o distorsió.

S'utilitza en memòria compartida pels workers i per seguir l'estat de cada tasca. D'aquest manera podem controlar i monitoritzar el progrés de les distorsions d'arxius i reprendre la tasca si cal.

4. TaskStatus

Enumeració que representa els possibles estats d'una tasca:

- TASK_EMPTY, TASK_RECEIVING, TASK_PROCESSING, TASK_COMPLETED, TASK_FAILED, etc.

Això ens simplifica molt el control de flux i la presa de decisions a cada fase (recepció, processament, reenviament...).

5. DistorsioStatus

Aquesta estructura és usada a Fleck per representar el progrés d'una tasca activa. Conté:

- progress: percentatge.
- estat: enum (ENVIANT, ESPERANT, etc.).

- mida_original, mida_distorsionat, bytes_enviats, bytes_rebuts, etc.

Així podem mostrar a l'usuari en temps real l'estat de cada tasca.

6. DistorsioArgs

Estructura temporal per passar paràmetres a un thread que inicia la distorsió. Inclou:

- IP i port del worker.
- Nom del fitxer.
- Factor de distorsió.
- worker_fd: socket de connexió.

És només per poder agrupar els paràmetres necessaris per iniciar la connexió a un worker de forma clara i segura.

7. Trama

Aquesta estructura fixa de 256 bytes representa la unitat bàsica de comunicació entre processos dins el sistema. Està empaquetada amb #pragma pack(1) per evitar padding i assegurar que s'envia exactament la mateixa estructura per xarxa.

Conté els següents camps:

- char type (1 byte): Indica el tipus de trama (per exemple, 0x01 per connexió, 0x03 per sol·licitud de distorsió, etc.).
- uint16_t data_length (2 bytes): Longitud del contingut efectiu dins el camp data.
- char data[245]: Camp de dades útils per la trama (pot contenir, per exemple, nom del fitxer, paràmetres, codis d'error...).
- uint16_t checksum: Comprovació d'integritat del contingut de la trama.
- uint32_t timestamp: Temps (en format Unix time) en què s'ha creat o enviat la trama.

Aquesta estructura permet una comunicació clara, robusta i controlada entre Gotham, Fleck, Harley, Enigma i Arkham. El checksum permet detectar corrupcions o errors de

transmissió. S'utilitza cada vegada que un procés envia o rep dades, que ho fa mitjançant trames d'aquest tipus.

2.2. Estratègia de concurrència adoptada per mòdul

Gotham

Gotham és el nucli coordinador. Actua com a servidor centralitzat que accepta connexions de Flecks (clients) i Workers (processadors de tasques), manté i actualitza l'estat actiu dels components, promou nous workers principals en cas de caigudes i comunica els components de forma segura i eficient.

Threads de separació funcional

Gotham crea dos threads principals (amb pthread_create):

- listener_fleck: exclusiu per gestionar connexions amb *Flecks*
- listener_worker: exclusiu per gestionar connexions amb *Workers*

Per què fer-ho amb threads separats?

- Cada tipus de client (Fleck o Worker) té protocols de comunicació diferents.
- Aïllar-los en fils separats permet: reutilització i mantenibilitat (cada listener és modular), evitar l'ús d'un bucle únic complicat i propens a errors, gestionar millor els recursos (cada thread fa poll() de forma independent), garantir respostes immediates a cada tipus de client, sense bloquejos creuats.

poll() intern per cada tipus de connexió

Dins de cada thread (listener_fleck, listener_worker) utilitzo poll() per controlar múltiples sockets de connexions entrants o actives, com ara:

- Nous Flecks que volen connectar-se.
- Heartbeats periòdics.
- Peticions de distorsió.
- Connexions de nous Workers o desconnexions.

Per què poll() i no select()?

- Escalabilitat: poll() és més eficient que select() quan tens molts sockets perquè no està limitat per FD_SETSIZE.
- Simplicitat: poll() usa una estructura d'array que és fàcil de gestionar i redimensionar amb realloc() (com faig al codi amb poll_fds).
- Evitació de busy waiting: gràcies als timeout en poll(), puc esperar events sense consumir 100% de CPU.
- Control de connexions mortes: combino poll() amb el camp last_activity[], que manté un registre de l'últim cop que es va rebre activitat. Si un socket supera el TIMEOUT_SECONDS, es tanca i s'elimina netament.

Mutexos per dades compartides

- worker_mutex i fleck_mutex: sincronitzen l'accés a les llistes globals workers[] i flecks[], que poden ser modificades des de qualsevol thread (afegir, eliminar, consultar).
- arkham_mutex: sincronitza l'escriptura a la pipe del logger per evitar que dos esdeveniments simultanis provoquin corrupció del log.

Aquests mutexos són imprescindibles perquè el model del sistema és *multi-threaded* i la concurrència accedint a dades compartides podria provocar condicions de carrera (ex: dos threads afegeixen o eliminen simultàniament un worker) o lectures inconsistents (ex: un Fleck rep una IP d'un worker que ja ha estat eliminat).

Logger a part (Arkham) amb fork() i pipe()

Gotham crea un procés fill (fork()) que rep missatges mitjançant una pipe anònima (pipe(log_pipe_fd)). El fill (Arkham) escriu els logs a logs.txt de forma síncrona.

Per què fer servir un procés separat?

- Aïllament de responsabilitats: escriure fitxers no bloqueja el fil d'execució principal.
- Evitem bloquejos: cap thread ha d'esperar I/O de disc.
- Robustesa: si Gotham cau, Arkham pot seguir escrivint (fins que la pipe es tanca).

El log_event() fa:

1. lock del mutex
2. genera el timestamp
3. escriu a la pipe
4. unlock

Això garanteix consistència, ordre i traçabilitat en els logs del sistema.

Senyal SIGINT per tancament net

Quan Gotham rep un Ctrl+C, la funció cleanup():

- tanca sockets,
- allibera memòria de workers[] i flecks[],
- mata el procés Arkham amb kill(SIGTERM),
- escriu al log que el sistema ha estat aturat netament.

Resum de decisions i alternatives

En el disseny de Gotham, s'ha optat per una arquitectura multithreaded amb separació funcional clara: es creen dos fils independents mitjançant pthread_create, un dedicat exclusivament a gestionar les connexions de Flecks i un altre per les connexions de Workers. Aquesta separació facilita el manteniment i escalabilitat del sistema, ja que cada tipus de client té un protocol de comunicació diferent i responsabilitats específiques. Aïllar-los en fils diferents permet modularitzar millor el codi i evitar interferències o bloquejos entre gestions que responen a fluxos molt diferents.

Dins de cada fil, s'utilitza poll() per controlar múltiples sockets simultàniament (tant nous intents de connexió com sockets actius ja registrats). La decisió d'utilitzar poll() en lloc de select() es basa en diverses raons: poll() no està limitat pel màxim nombre de descriptors (FD_SETSIZE) com sí que ho està select(), i la seva interfície amb arrays dinàmics permet redimensionar fàcilment l'estructura amb realloc() segons les necessitats. Això fa que el sistema sigui més fàcil d'adaptar a increments de càrrega i a un nombre variable de connexions entrants.

Per gestionar les dades compartides entre aquests dos fils (com ara les llistes de Flecks i Workers registrats), s'utilitzen mutexos específics (fleck_mutex, worker_mutex) que garanteixen que no es produeixin condicions de carrera. S'ha descartat l'ús de rwlock (lectura/escriptura) perquè l'accés concurrent de lectura no és prou freqüent ni intensiu per justificar la seva complexitat. Un simple mutex ofereix prou rendiment i simplicitat.

Per registrar els esdeveniments de manera persistent i ordenada, Gotham crea un procés fill anomenat Arkham mitjançant fork(), que s'encarrega exclusivament d'escriure els logs a un fitxer. Aquesta elecció, en lloc de fer-ho amb un thread, es basa en què un procés separat garanteix un aïllament complet: si Arkham pateix errors o bloquejos d'entrada/sortida, no pot afectar el funcionament del procés pare. A més, l'ús d'un fork() simplifica la gestió de concurrència dins del logger, ja que no comparteix memòria amb la resta de fils de Gotham i, per tant, no cal protegir accés al fitxer amb mutexos complexos dins d'Arkham. Això afegeix robustesa, modularitat i traçabilitat al sistema, assegurant que el registre d'esdeveniments sigui segur fins i tot si Gotham falla.

Per detectar desconnexions de clients inactius, s'ha implementat un sistema lleuger basat en last_activity[], una estructura que emmagatzema l'última interacció registrada amb cada socket. A cada iteració de poll(), si es detecta que una connexió ha superat un cert llindar d'inactivitat (TIMEOUT_SECONDS), es tanca i es neteja. Aquesta estratègia evita la necessitat de fer heartbeats periòdics des de Gotham (que haurien d'enviar-se activament a tots els clients), estalviant recursos i simplificant el protocol. En canvi, s'exigeix que siguin els Flecks i Workers qui enviïn els heartbeats, i Gotham simplement en monitoritza l'absència.

Aquesta combinació de poll(), separació per fils, ús moderat de mutexos, pipes i tractament actiu de connexions inactives ha permès construir un sistema escalable, robust i eficient, que respon bé sota càrrega i manté la modularitat necessària per facilitar futures ampliacions (com suportar múltiples Gotham, logs externs, etc.).

Workers (Harley / Enigma)

Els Workers són els encarregats d'**efectuar la distorsió real dels fitxers**, ja siguin multimèdia (Harley) o text (Enigma). Cada Worker pot rebre múltiples peticions de Fleck,

distorsionar en paral·lel, comunicar-se amb Gotham (heartbeat, registre, desconnexió…) i gestionar memòria compartida per persistir l'estat de les tasques.

Un únic procés amb diversos threads (multithreaded)

Cada Worker funciona com un únic procés multithreaded que crea diversos fils segons la seva responsabilitat:

- **Thread de heartbeats:** envia periòdicament trames 0x12 a Gotham per mantenir activa la connexió.
- **Thread d'escolta de Gotham:** rep trames de control, com 0x08 (promoció a principal), 0x07 (desconnexió), etc.
- **Thread d'escolta a Fleck:** només actiu si el Worker ha estat promogut a principal; accepta connexions entrants.
- **Thread per cada distorsió:** quan un Fleck envia una petició DISTORT, es crea un thread específic per gestionar-la de forma completa (recepció, processament, enviament).

Per què s'ha fet així?

Permet paral·lelitzar múltiples distorsions simultànies de manera aïllada. A més, aprofita la simplicitat de pthread_create() + pthread_detach() per no haver d'esperar explícitament la finalització. Així evitem bloquejos del procés principal en operacions costoses o lentes (ex: compressió).

select() per escoltar sockets sense bloqueig

S'utilitza select() per vigilar el socket amb Gotham (rebre trames) I el socket d'escolta de Fleck (acceptar noves connexions, si és principal).

Per què select() i no poll()?

- El nombre de sockets a gestionar per cada Worker és petit, així que select() és suficient i més lleuger que poll().
- La combinació amb sockets O_NONBLOCK i select() amb timeout permet:
    - *polling suau*: no fer bucles actius que saturin CPU.
    - detectar activity i sortir del bucle si keep_running = 0 o es rep SIGINT.

Thread per cada tasca de distorsió

Quan arriba una petició DISTORT, es crea un thread (detached) que:

- rep el fitxer (trames 0x05),
- distorsiona (SO_compressX() o processar_text()),
- envia metadades + fitxer de tornada,
- envia resposta CHECK_OK o CHECK_KO.

Per què fer un thread per distorsió?

- Aïlla completament cada petició, fent el sistema més robust davant errors o bloquejos parcials.
- Permet aprofitar els recursos del sistema i processar diversos fitxers en paral·lel.
- Simplifica el codi perquè cada thread només gestiona una única tasca, evitant estat compartit complex.
- Cura presa:
    - Tots els threads són pthread_detach per no fer join().
    - Però es manté un recompte actiu active_threads protegit per threads_mutex per saber si queda cap tasca en curs (important per tancament net).

Aquest model pot generar massa threads si hi ha molts Flecks actius alhora. A causa del límit de processos/threads per usuari (ulimit -u), això pot provocar errors com *Resource temporarily unavailable*. S'hauria d'adaptar el disseny per evitar saturació massiva. Igual que a Gotham, una possible solució és fer ús de poll() per gestionar múltiples sockets de Flecks des d'un únic thread escoltador. Això permetria limitar la creació de nous fils, fent el sistema més escalable.

Memòria compartida amb mmap

Cada Worker utilitza memòria compartida per guardar l'estat de les tasques (estructura TaskMetadata).

- Inicialització: shm_open, ftruncate, mmap.
- Protecció: accés sincronitzat amb shared_mem_mutex.
- Dinamisme: si no hi ha espai, crido expandir_memoria_compartida().

Això permet que una altra instància pugui reprendre una tasca (si un worker cau, l'altre pot veure l'estat parcial).

Mutexos i sincronització

Hi ha alguns punts del sistema que són crítics quan hi ha diversos fils funcionant alhora. Aquests punts cal protegir-los per evitar problemes com condicions de carrera o dades inconsistents.

Per exemple, la memòria compartida (shared_mem) és accedida per més d'un fil, i per tant utilitzem un mutex anomenat shared_mem_mutex per assegurar que només un fil hi pugui llegir o escriure a la vegada. Això evita que dues operacions simultànies provoquin corrupció de dades.

També fem servir un altre mutex, anomenat threads_mutex, per protegir l'accés al recompte de fils actius (active_threads). Si més d'un fil intenta modificar aquest número alhora, podríem tenir un recompte incorrecte. Amb el mutex, evitem aquesta situació.

Gestió de senyals (SIGINT)

Quan el Worker rep SIGINT, fa:

- Cancel·la els fils escoltadors (pthread_cancel).
- Tanca tots els sockets oberts.
- Allibera la memòria compartida (shm_unlink).
- Fa exit() després d'haver netejat tot.

Resum de decisions i alternatives

En el disseny dels Workers, s'ha optat per un model multithreaded en què un únic procés gestiona totes les funcionalitats, delegant les diferents responsabilitats a fils separats. Cada Worker manté diversos threads en paral·lel: un per enviar heartbeats a Gotham, un altre per escoltar les seves trames de control, i un tercer per escoltar connexions entrants de Fleck si ha estat promogut a principal. Quan un Fleck es connecta i envia una petició de distorsió, es crea un thread dedicat per gestionar aquesta tasca de principi a fi. Això inclou la recepció del fitxer, el processament (compressió o manipulació de text), i l'enviament del fitxer distorsionat de tornada.

S'ha decidit fer un thread per cada tasca per maximitzar la concurrència i garantir que els problemes en una distorsió concreta no afectin la resta. A més, com que la compressió de fitxers pot ser una operació costosa, delegar-la a un thread aïllat evita

bloquejos o lentitud en el thread principal. Aquest model simplifica el flux de control i manté la claredat del codi, ja que cada fil treballa de forma autònoma.

Per escoltar activitat en els sockets de Gotham i Fleck, s'ha fet servir select(). En aquest cas concret, poll() no ofereix avantatges significatius, ja que cada Worker només manté oberts pocs sockets (normalment dos o tres). select() resulta més lleuger i senzill per a aquest escenari. La combinació de select() amb sockets no bloquejants i un petit timeout permet evitar bucles actius que consumirien CPU innecessàriament, mantenint una espera eficient d'esdeveniments.

Tot i així, el model "thread-per-task" presenta una limitació important: si hi ha molts Fleck simultanis enviant fitxers, es poden arribar a crear massa fils. Això pot provocar errors del tipus "Resource temporarily unavailable", especialment si el sistema té un límit baix de processos per usuari (ulimit -u). És per això que, com a proposta de millora, s'hauria de considerar la implementació d'un sistema amb poll() centralitzat, similar al de Gotham, que permeti gestionar múltiples connexions sense generar un nou thread per cadascuna. Aquesta alternativa faria el sistema més escalable i resistent a situacions de molta càrrega.

Pel que fa a l'estat de les tasques, s'ha implementat un sistema de memòria compartida mitjançant shm_open i mmap, que permet persistir informació estructurada (TaskMetadata). Aquesta memòria es pot llegir i escriure des de qualsevol fil del Worker, i també des d'un altre Worker que pugui reprendre les tasques en cas de caiguda. Per evitar condicions de carrera, es protegeix l'accés a aquesta memòria amb el mutex shared_mem_mutex.

A més, es manté un recompte global de fils actius (active_threads), que serveix per saber si encara hi ha tasques en curs. Aquest recompte es modifica dins blocs protegits per un altre mutex (threads_mutex), assegurant coherència i permetent que el tancament del Worker (per exemple, en rebre SIGINT) sigui ordenat i net.

Finalment, el sistema captura la senyal SIGINT per fer una neteja controlada: cancel·la els fils escoltadors, tanca sockets, allibera la memòria compartida i finalitza l'execució. Aquesta estructura garanteix una terminació robusta, evitant pèrdua de dades o memòria mal gestionada.

En conjunt, s'ha apostat per un disseny clar, modular i paral·lel, que dóna bon rendiment per un nombre moderat de peticions. Però també s'han identificat possibles colls d'ampolla i camins d'evolució futura per millorar-ne l'escalabilitat i resistència sota càrrega elevada.

Fleck

Fleck és el procés client amb què interactua l'usuari. Té com a objectiu principal connectar-se amb Gotham, sol·licitar distorsions de fitxers i gestionar tota la comunicació amb els Workers, incloent l'enviament i recepció de dades. A diferència de Gotham i els Workers, Fleck no és un servidor sinó un client actiu que inicia les connexions.

Separació funcional per fils

Fleck no escolta connexions, però sí que gestiona comandes de l'usuari que poden trigat molt a completar-se (com una distorsió de vídeo o àudio). Per això, quan l'usuari escriu una comanda DISTORT, es crea un nou thread exclusiu per executar-la. Aquest fil pot durar molts segons, però no bloqueja el prompt principal de Fleck. Això permet executar comandes com CHECK STATUS o CLEAR ALL mentre una distorsió és en curs.

Per què aquesta decisió?

- Permet concurrència sense necessitat de gestionar múltiples sessions.
- És senzill i compatible amb una arquitectura client (no cal cap sistema de polling).
- La gestió de recursos és clara: el fil es detach immediatament i allibera memòria en acabar.

Si es volgués escalar més (múltiples distorsions simultànies per tipus), es podria encapsular cada distorsió amb un context més rígid i controlar les connexions de manera més estructurada (ex. amb descriptors multiplexats o epoll), però per aquest projecte, el model actual és eficient i senzill.

Heartbeat: connexió viva amb Gotham

Fleck envia un heartbeat cada 5 segons a Gotham mitjançant un fil dedicat (heartbeat_gotham). Si Gotham cau, aquest fil és el primer a detectar-ho i crida parar_fleck() per netejar recursos.

Per què aquesta decisió?

- El heartbeat garanteix que Gotham sàpiga que Fleck segueix viu.
- També permet a Fleck detectar una pèrdua de connexió abans d'una petició fallida.
- És millor fer això en un fil separat que dins del bucle de comandes, ja que permet independència.

Gestió de concurrència amb mutex

Fleck utilitza dos pthread_mutex_t:

- fd_mutex: protegeix l'accés als descriptors globals harley_fd i enigma_fd.
- status_mutex: protegeix la llista dinàmica de distorsions en curs (DistorsioStatus).

Aquest últim és fonamental per garantir que l'estat de progrés sigui coherent entre fils, ja que múltiples distorsions poden estar actives i accedir-hi alhora.

Sense poll ni select: per què no calen

Fleck no fa servir poll() ni select() perquè:

- Només es connecta a un Worker a la vegada per tipus (media / text).
- El socket és síncron i amb SO_RCVTIMEO per evitar bloquejos indefinits.
- La major part del temps, Fleck està llegint/escrivint seqüencialment.

Per tant, l'ús de poll() o select() no aportaria beneficis clars en aquest cas. Això sí, si es volgués implementar recepció asíncrona (per ex. Fleck rebent notificacions de múltiples workers a la vegada), sí caldria un model més sofisticat.

Tolerància a errors i reassignació automàtica

Una de les millors característiques de Fleck és que detecta si un Worker ha caigut a qualsevol fase (enviament, espera, recepció) i es recupera automàticament:

- Sol·licita un nou Worker a Gotham.
- Torna a obrir el fitxer i el reenvia des del punt anterior.
- Valida la connexió de nou.

Això fa Fleck un client resilient. No cal que l'usuari intervingui; el sistema es recupera de forma transparent.

Millora proposada: encapsulació de connexions

Ara mateix, harley_fd i enigma_fd són globals, amb gestió manual. Es podria encapsular cada connexió en una estructura que inclogués:

- Tipus (media / text)
- Estat
- Últim heartbeat rebut
- Fase actual

Resum de decisions i alternatives

Al dissenyar Fleck, s'han pres diverses decisions orientades a garantir una bona experiència d'usuari, mantenint el sistema simple però robust. Per començar, s'ha optat per crear un nou fil d'execució (thread) cada cop que l'usuari introdueix una comanda DISTORT. Aquesta elecció permet que les distorsions es realitzin en paral·lel sense bloquejar l'entrada de comandes. Això fa que l'usuari pugui continuar consultant l'estat o netejant distorsions mentre una està en curs, afavorint l'ús interactiu i fluid del sistema.

Per mantenir la connexió amb Gotham, s'ha implementat un fil separat que envia heartbeats cada 5 segons. Aquesta estratègia permet detectar si Gotham ha caigut sense haver d'esperar a una petició fallida. És una forma senzilla però efectiva de mantenir la connexió viva i actuar ràpidament si el servidor no respon.

En relació amb la gestió de recursos, Fleck no utilitza ni select() ni poll(), ja que només manté una connexió activa amb cada Worker alhora (una per tipus: media o text). Com que les operacions d'escriptura i lectura són seqüencials i les connexions es controlen individualment, aquest disseny resulta prou eficient i fàcil de mantenir. Si en el futur es volgués permetre múltiples distorsions del mateix tipus, es podria considerar encapsular cada connexió i fer servir poll() per gestionar-les millor, però per l'escenari actual no és necessari.

La tolerància a errors és un altre dels punts forts de Fleck. Si un Worker cau en qualsevol fase (enviament, espera o recepció), s'intenta reconectar automàticament amb un nou Worker sol·licitat a Gotham. El sistema reobre el fitxer original i continua des d'on s'havia quedat, permetent una recuperació automàtica sense que l'usuari hagi d'intervenir. Aquesta funcionalitat aporta robustesa i una experiència molt més professional.

Finalment, la informació de l'estat de les distorsions es guarda en una estructura dinàmica protegida amb un mutex (status_mutex). Això assegura que diversos fils no accedeixin alhora a la mateixa estructura, evitant condicions de carrera i permetent mostrar l'estat en qualsevol moment amb CHECK STATUS.

En conjunt, s'ha triat un enfocament seqüencial però concurrent quan cal, que permet un control clar dels recursos i una interacció fluida per part de l'usuari. El sistema es manté simple, robust i fàcil d'escalejar en cas que es volguin afegir funcionalitats més avançades en un futur.

2.3. Gestió de memòria compartida

Els processos Harley i Enigma comparteixen una àrea de memòria comuna per gestionar l'estat de les tasques de distorsió. Aquesta memòria es crea i accedeix mitjançant la crida a shm_open("/worker_shared_mem", ...), i permet que diversos workers cooperin o es rellevin en la gestió de fitxers, garantint la persistència i continuïtat de les tasques.

Estructura de la memòria compartida

La memòria compartida conté:

- Un enter inicial: int worker_count, que indica el nombre de workers actius en aquell moment.
- Un array dinàmic de TaskMetadata, que conté informació sobre cada tasca activa: nom de fitxer, mida total, bytes rebuts/enviats, estat, checksum, etc.

Aquesta estructura permet que diversos workers accedeixin simultàniament a la informació sobre les tasques en curs, i col·laborin en la seva gestió si és necessari.

Ampliació dinàmica amb mremap

L'array de TaskMetadata pot créixer a mesura que es reben noves tasques. Això s'aconsegueix mitjançant la funció mremap, que redimensiona la memòria compartida

de forma segura, preservant el contingut existent. Es descarta fixar un límit màxim de tasques per evitar desaprofitar memòria o limitar el sistema artificialment. El creixement dinàmic permet escalar segons les necessitats reals d'ús.

Sincronització d'accés

L'accés a la memòria es protegeix mitjançant:

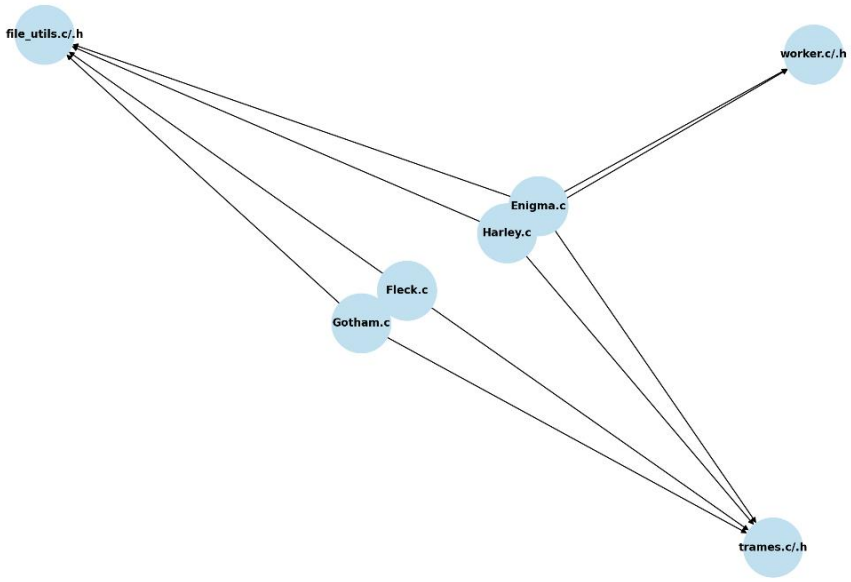
- shared_mem_mutex: un mutex global per operacions crítiques com afegir, eliminar o modificar tasques.

Això garanteix la coherència i seguretat de les dades, fins i tot amb múltiples threads treballant en paral·lel.

Persistència entre Workers

Quan un worker cau o és eliminat, la informació sobre les tasques que gestionava es manté a la memòria compartida. Això permet que altres workers o un nou worker promogut a principal pugui reprendre la tasca sense pèrdua de dades.

2.4. Modularització del codi



El projecte *Mr. J. System* s'ha estructurat de forma modular i clara per facilitar la mantenibilitat, la reutilització de codi i el desenvolupament col·laboratiu. La modularització s'ha aplicat tant a nivell de fitxers com a nivell de funcionalitat.

Estructura de carpetes

El projecte es divideix en les següents carpetes principals:

- **Fleck/**: conté el codi del client principal que sol·licita distorsions.
- **Gotham/**: implementa el servidor central que coordina Flecks i Workers.
- **Worker/**: conté Harley i Enigma, els processos que realitzen les distorsions. També inclou worker.c/.h, que agrupa funcionalitats comunes.
- **Utils/**: biblioteca compartida amb utilitats generals i definicions comuns (trames.c, file_utils.c, sys_params.h, etc.).

Aquesta divisió reflecteix clarament el **model de processos distribuïts** i permet reutilitzar codi comú entre Harley i Enigma, així com evitar duplicitats a Gotham o Fleck.

Llibreries compartides

S'han creat diversos arxius .h per encapsular funcionalitats transversals i compartir-les entre components:

worker.h

Agrupa les funcions comunes a Harley i Enigma, entre elles:

- Gestió de tasques: recepció de fitxers (gestionar_recepcio_fitxer), detecció de desconnexió, enviament del fitxer distorsionat (enviar_fitxer_processat).
- Gestió de memòria compartida: inicialització (inicialitzar_memoria_compartida), cerca (buscar_tasca_per_nom), ampliació (expandir_memoria_compartida), i neteja.
- Connexió amb Gotham: inclou funcions de connexió, reconnexió i notificació de desconnexió.

Aquesta llibreria evita la duplicació de codi entre Harley i Enigma i permet adaptar fàcilment lògica comuna.

file_utils.h

Inclou utilitats generals com:

- write_message(): per escriure missatges a un fitxer descriptor.

- calcular_md5() i verificar_md5sum(): per garantir la integritat dels fitxers.
- Funcions de validació (validate_ip, trim_whitespace) i lectura segura (safe_read()).
- Parsers de configuració (parse_gotham_config, parse_worker_config, parse_fleck_config).

Aquest fitxer actua com una caixa d'eines comuna per a tots els processos del sistema.

trames.h

Defineix l'estructura del protocol de trames i les operacions associades:

- Trama: estructura compactada de 256 bytes.
- inicialitzar_trama, validar_trama, generate_checksum: per gestionar l'enviament i validació.
- Funcions específiques de connexió, distorsió i error (send_connection_request_to_gotham, send_error_notification, etc.).

Gràcies a aquest fitxer, tots els processos treballen amb el mateix protocol de forma centralitzada i segura.

Beneficis de la modularització

La modularització del projecte ha permès assolir diversos beneficis clau. En primer lloc, la reutilització del codi s'ha aconseguit centralitzant la definició de funcions compartides en arxius específics com worker.h, file_utils.h o trames.h, evitant així duplicació i facilitant l'evolució del sistema. A més, la separació de responsabilitats ha estat clara: cada mòdul (Fleck, Gotham, Harley, Enigma, Utils) s'encarrega d'una tasca concreta dins del sistema, seguint una arquitectura coherent i fàcil d'entendre.

Gràcies a aquesta organització, cada component pot ser compilat, provat i depurat de forma independent, cosa que en facilita enormement el manteniment i la detecció d'errors. Finalment, el sistema ha estat dissenyat pensant en l'escalabilitat: si en un futur es volgués afegir un nou tipus de Worker o incorporar noves funcionalitats, es podrien integrar fàcilment aprofitant els mòduls i llibreries ja existents.

2.5. Funcionalitats opcionals implementades

D'entre les funcionalitats descrites a l'enunciat, l'única considerada opcional era la implementació del mecanisme de heartbeat per detectar la caiguda de processos. Tot i no ser obligatori, aquest sistema s'ha implementat completament entre Gotham, els Workers i Fleck.

Els heartbeats consisteixen en trames de tipus 0x12 amb DATA_LENGTH = 0, que s'envien de manera periòdica (cada 5 segons) per confirmar que un procés continua actiu. Això permet que Gotham detecti caigudes de Workers o de Flecks, i actuï en conseqüència (per exemple, eliminant-los de les estructures internes o promovent un nou Worker principal). També s'utilitzen en sentit invers per detectar si Gotham ha deixat de respondre.

La seva implementació es va considerar rellevant perquè aporta realisme, estabilitat i tolerància a fallades al sistema. En sistemes distribuïts reals, el monitoratge de la salut dels processos és essencial per garantir un servei continu i fiable, especialment quan hi ha múltiples usuaris i recursos compartits.

Gràcies als heartbeats, Mr. J. System pot reaccionar de manera automàtica i ordenada davant caigudes, mantenint la consistència del sistema i minimitzant la pèrdua d'informació.

3. Problemes observats i solucions

Durant el desenvolupament del projecte Mr. J. System s'han presentat diversos reptes tècnics i organitzatius que han condicionat tant la planificació com la implementació de les funcionalitats. Aquest apartat recull les dificultats més rellevants i les solucions adoptades o identificades.

Tot i que la pràctica estava pensada per fer-se en parelles, el meu company va deixar de respondre poc abans de l'entrega del checkpoint de la fase 2, fet que em va obligar a assumir totes les responsabilitats del projecte, tant a nivell de disseny com de programació, debugging, testing i redacció de la memòria. Això va representar una càrrega de treball considerable, sobretot tenint en compte la complexitat d'un sistema distribuït amb concurrència real, sockets TCP, memòria compartida dinàmica i un protocol de comunicació propi. Davant aquesta situació, vaig haver de replanificar completament les tasques, prioritzant els elements essencials per garantir el funcionament correcte del sistema i deixant en segon pla certes optimitzacions o funcionalitats avançades no estrictament obligatòries. Tot i així, es va aconseguir implementar totes les fases requerides, incloent una versió funcional i estable de Gotham, Harley, Enigma i Fleck, amb suport per heartbeats, promoció automàtica de workers principals i gestió segura i dinàmica de la memòria compartida.

Es van detectar casos de doble bloqueig, especialment en funcions com netejar_connexio_per_error, on es feia pthread_mutex_lock() sobre un recurs que ja estava protegit en el context superior. Aquesta duplicació podia provocar deadlocks o esperes indefinides. Es va revisar tota la gestió de locks, centralitzant el control en el context principal i indicant amb comentaris clars quan una funció espera rebre un recurs ja bloquejat. Això ha permès simplificar i fer més robust el flux d'execució.

Pel que fa a la concurrència, un dels casos límit que no s'ha pogut resoldre completament és quan dos Flecks sol·liciten alhora la distorsió d'un mateix fitxer amb el mateix nom. El sistema actual no protegeix contra aquest cas, i això pot provocar conflictes dins la memòria compartida, ja que dos Workers podrien accedir o sobreescriure la mateixa entrada TaskMetadata, generant condicions de carrera o comportaments indeterminats.

Tot i que per enviar el fitxer original des de Fleck no es va considerar necessari canviar-ne el nom, ja que es fa en mode lectura i no hi ha modificació directa, sí que es va detectar un problema al rebre el fitxer distorsionat del Worker: si dos Flecks reben alhora un fitxer amb el mateix nom, un pot sobreescriure el resultat de l'altre. Per evitar-ho, es va implementar una solució en què **el fitxer distorsionat rebut pel Fleck es guarda amb un nom únic que incorpora el pid del procés**. D'aquesta manera, es garanteix que cada instància de Fleck guarda el seu resultat de forma independent, evitant sobreescriptures locals.

A nivell de Worker, però, aquest cas continua sense estar resolt. El conflicte que es genera quan dues peticions arriben amb el mateix file_name en paral·lel no té, de moment, una solució clara dins del sistema implementat. Es va valorar la possibilitat d'afegir una validació prèvia que comprovés si ja existeix una tasca activa amb aquell nom, però això planteja nous reptes: com sincronitzar correctament l'accés a la memòria compartida entre múltiples threads sense afectar el rendiment, o com gestionar possibles col·lisions amb noms legítimament iguals però provinents de contextos independents. El temps limitat i la priorització d'altres aspectes més crítics del funcionament del sistema no han permès explorar i implementar una solució definitiva per aquest cas, que es manté com un **problema obert** i una **possible millora futura**.

Tot i que Gotham implementa poll() per gestionar múltiples sockets de manera eficient, els Workers actualment gestionen les connexions amb accept() seqüencial i llancen un thread per cada Fleck. Això implica que no poden monitorar diversos sockets simultàniament i limita l'escalabilitat. No vaig tenir temps d'implementar poll() a Harley i Enigma, però una solució futura implicaria refactoritzar els seus bucles principals per operar amb conjunts de descriptors, associant cada connexió amb una estructura d'estat i evitant la creació massiva de threads.

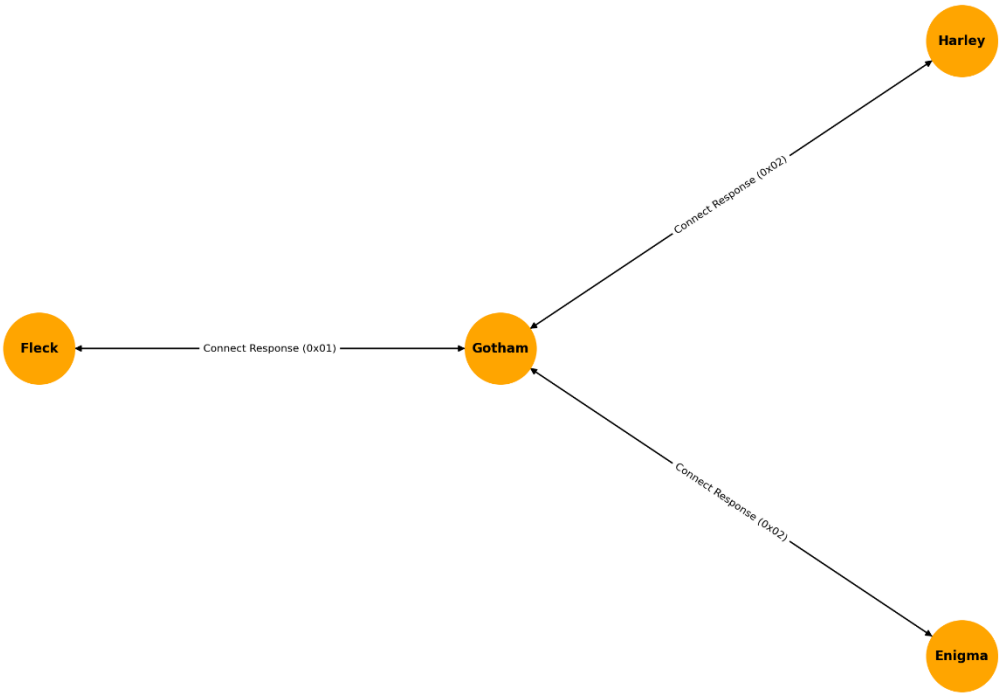
Finalment, la implementació dels heartbeats també va comportar certs desafiaments. Inicialment, processos actius eren marcats com a caiguts perquè els intervals de comprovació eren massa estrictes o asimètrics. Es va ajustar curosament la temporització, establint un interval de 5 segons per a l'enviament i validació, i es va utilitzar gettimeofday() per comparar timestamps i assegurar una tolerància adequada. També es van fer proves simulant caigudes reals (tancant processos manualment) per

verificar que Gotham gestionava correctament les eliminacions i promocions de manera segura i controlada.

En conjunt, tot aquest procés ha implicat una depuració contínua i una adaptació constant a nous reptes tècnics, sempre prioritzant la robustesa i estabilitat del sistema. Tot i les limitacions de temps i recursos, el sistema final és funcional, escalable i preparat per gestionar múltiples distorsions simultànies amb una arquitectura modular i resistent.

4. Diagrames de Flux

Connexió inicial de Fleck amb Gotham i dels Workers amb Gotham



Aquest diagrama representa el flux d'establiment de connexions entre els diferents processos (Fleck, Harley, Enigma) amb el servidor central Gotham.

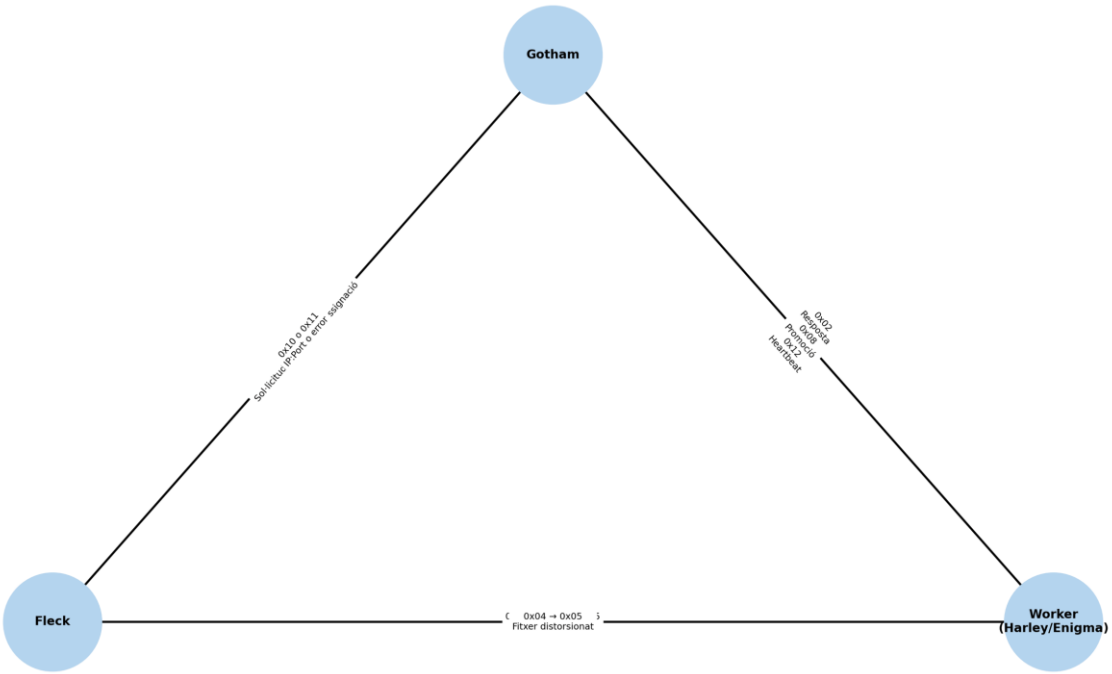
1. Connexió de Fleck amb Gotham
    - Fleck → Gotham
        - Trama de petició:
            - Tipus: 0x01
            - Contingut (DATA): <userName>&<IP>&<Port>
    - Gotham → Fleck
        - Trama de resposta:
            - Tipus: 0x01
            - Si la connexió és exitosa: DATA buit (DATA_LENGTH = 0)

        - Si la connexió falla: DATA = "CON_KO"

2. Connexió de Workers (Harley / Enigma) amb Gotham

    - Worker → Gotham
        - Trama de petició:
            - Tipus: 0x02
            - Contingut (DATA): <workerType>&<IP>&<Port>
    - Gotham → Worker
        - Trama de resposta:
            - Tipus: 0x02
            - Si la connexió és acceptada: DATA buit
            - Si és rebutjada: DATA = "CON_KO"

Gestió de Distorsió



Aquest diagrama mostra el flux d'una petició de distorsió des de **Fleck** cap a **Gotham** i la seva gestió amb els Workers.

1. Comunicació entre Fleck i Gotham
    - Fleck → Gotham
        - Trama 0x10 per a petició inicial o 0x11 per a reassignació.
        - Contingut: <mediaType>&<fileName>
    - Gotham → Fleck
        - Trama 0x10 o 0x11
        - Resposta amb DATA = <IP>&<Port> del Worker o bé "DISTORT_KO" / "MEDIA_KO" en cas d'error.
2. Comunicació entre Fleck i el Worker assignat
    - Fleck → Worker
        - Trama 0x03: Metadades del fitxer (nom, mida, hash, factor)
        - Trama 0x04: Confirmació de mida i MD5
        - Trames 0x05: Enviament del contingut del fitxer en blocs
    - Worker → Fleck
        - Trama 0x04: Mida i MD5 del fitxer distorsionat
        - Trames 0x05: Fitxer distorsionat enviat en blocs

3. Comunicació entre Worker i Gotham

- **Worker → Gotham**
    - Trama 0x02: Petició de connexió inicial
    - Trama 0x12: Heartbeat periòdic indicant que està actiu
- **Gotham → Worker**
    - Trama 0x02: Confirmació o rebuig de la connexió
    - Trama 0x08: Promoció a Worker principal
    - Trama 0x12: Sol·licitud de heartbeat

5. Estimació temporal

Activitat	Hores dedicades
Investigació	15 h
Disseny	15 h
Implementació	50 h
Testing	25 h
Documentació	10 h
Total	**115 h**

6. Conclusions i propostes de millora

El desenvolupament del projecte *Mr. J. System* ha estat una experiència tècnica i personalment exigent, però també molt enriquidora. Afrontar la implementació d'un sistema distribuït complex, amb múltiples processos que cooperen mitjançant sockets TCP, gestió de memòria compartida, multithreading i un protocol binari propi, ha suposat un salt qualitatiu en la meva comprensió del funcionament intern d'aquest tipus d'arquitectures.

Tot i que el projecte estava pensat per ser realitzat en parella, les circumstàncies em van portar a desenvolupar-lo pràcticament sola. Això va suposar una càrrega de treball molt elevada, però també m'ha permès tenir una visió completa de tot el sistema: des del disseny del protocol i la modularització del codi fins a la gestió d'errors, concurrència, sincronització i traçabilitat. He après a prendre decisions sota pressió, a prioritzar funcionalitats clau, i a buscar solucions eficients amb el temps disponible. Malgrat les limitacions, considero que he assolit els objectius de la pràctica i he lliurat un sistema funcional, estructurat i preparat per gestionar múltiples distorsions simultànies.

Una de les millores més importants que hauria d'afrontar-se en una propera iteració és la implementació de poll() als Workers (Harley i Enigma). Actualment, cada Worker només pot gestionar una connexió de Fleck per thread, llançant un nou thread per cada connexió entrant mitjançant accept() i pthread_create(). Aquest enfocament és funcional però no escalable, especialment en entorns on diversos Flecks poden sol·licitar distorsions alhora. La inclusió de poll() (o select()) permetria gestionar múltiples sockets oberts dins un mateix thread, supervisant l'activitat de cada connexió i decidint quan llegir o escriure segons la disponibilitat del descriptor. Això reduiria l'ús de recursos, facilitaria la gestió de temps d'espera i milloraria l'eficiència general del sistema.

Un altre aspecte crític pendent de resolució és el problema de concurrència quan dos o més Flecks demanen la distorsió d'un fitxer amb el mateix nom al mateix temps. Aquesta situació pot provocar col·lisions a nivell de memòria compartida, ja que els Workers poden acabar gestionant la mateixa TaskMetadata de forma simultània, amb conseqüències com corrupció de dades, duplicació d'estats o inconsistències en el registre de tasques. A nivell de Fleck, el problema es va mitigar fent que el fitxer

distorsionat rebut es desi amb un nom únic que incorpora el pid, evitant sobreescriptures locals. Tot i això, el conflicte més profund —el de concurrència dins la memòria compartida del Worker— no té una solució implantada actualment. Es van valorar alternatives com bloquejos per nom de fitxer o validacions prèvies a l'acceptació de la connexió, però no es va poder implementar cap d'elles per manca de temps i pel fet que cap enfocament resultava del tot satisfactori sense introduir nova complexitat o nous punts de bloqueig. Aquest problema roman com una assignatura pendent clau per garantir la consistència en escenaris de concurrència alta.

A més d'aquests dos punts centrals, hi ha altres millores que podrien aportar una major robustesa, mantenibilitat i control del sistema. Per exemple, automatitzar proves de càrrega i de casos límit mitjançant scripts o frameworks de testing permetria validar de manera sistemàtica la resistència del sistema davant caigudes inesperades, connexions intermitents o situacions de concurrència elevada. També seria molt útil una gestió més detallada dels recursos, com ara sockets oberts, memòria dinàmica i threads actius, per detectar i evitar fuites o situacions de sobrecàrrega, especialment en entorns amb molts processos simultanis.

Finalment, una millora clara seria l'ampliació del sistema de logging i de missatges d'estat, de manera que el sistema pogués oferir més informació en temps real sobre l'activitat dels processos, les tasques en curs i els possibles colls d'ampolla. Això facilitaria enormement tant el debugging com la supervisió en escenaris de producció simulats, i ajudaria a detectar patrons de fallida o comportaments anòmals que ara només poden identificar-se manualment.

En conclusió, considero que *Mr. J. System* ha estat un projecte complet, amb una implementació funcional de totes les parts nuclears i una arquitectura clara, modular i coherent. Tot i això, com qualsevol sistema real, sempre hi ha marge de millora: especialment en termes d'escalabilitat, control de concurrència i cobertura de casos especials. A nivell d'aprenentatge, ha estat una experiència que m'ha obligat a prendre decisions tècniques sota pressió, a entendre a fons com funcionen les comunicacions entre processos i a valorar la importància d'una gestió acurada dels recursos i de la sincronització.

7. Explicació noms dels processos

Pel que fa a la temàtica de **Mr. J. System**, aquesta pràctica està inspirada en l'univers de **Batman**, concretament en el personatge del Joker (Arthur Fleck) i altres figures rellevants com Harley Quinn i l'Enigma (Riddler). Aquesta relació es desprèn de diversos elements:

- **Arthur Fleck** és el nom original del Joker en la pel·lícula *Joker* (2019). Aquest és el personatge central del sistema, que busca implementar un món amb la seva "realitat distorsionada".
- **Harley** i **Enigma** són els noms dels *workers* encarregats de processar peticions de distorsió de *media* i text, respectivament. Aquests noms fan referència a Harley Quinn i al Riddler, personatges icònics de l'univers de Batman.
- **Gotham**, el servidor central, representa la ciutat fictícia de l'univers de Batman. Aquest servidor és el cor del sistema i actua com a punt de connexió per als clients (Fleck) i els *workers*.

8. Bibliografia

- Canaleta, X. (2020). *Guia d'Estil de Sistemes Operatius*. La Salle, Universitat Ramon Llull.
- Canaleta, X. (2020). *Pràctica SO 2024-25*. La Salle, Universitat Ramon Llull.
- Canaleta, X. (2020). *Documents de testing (Fases 1 a 4)*. La Salle, Universitat Ramon Llull.
- Canaleta, X. (2020). *Capítol 4.5.1: Semàfors*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 4.1: Pipes i Memòria Compartida*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 4.2: Cues de Missatges*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 5.1: File Descriptors i Senyals*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2021). *Tema 5.2: Forks*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 5.3: Threads*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 5.4: Sockets*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2021). *Tema 5.5: Pipes*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 5.6: Memòria Compartida*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2021). *Tema 5.7: Cues de Missatges*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.